

End-to-End Lineage for AI-Era Data Systems

Data | Feature | Model | Prompt | Agent | Knowledge Lineage — Impact Analysis & Compliance

A complete study of lineage architectures spanning tables to autonomous agents — covering platform comparison, impact and root-cause analysis, compliance reporting, and AI traceability.

Data Architects

Platform Engineers

ML Engineers

AI Governance Teams

Compliance Teams

Risk Teams

Agent Developers

Covers: Six Lineage Layers • Platform Comparison • Impact & Root-Cause Analysis

Table of Contents

Executive Summary	3
The Six-Layer Lineage Model	5
1 — Data Lineage	7
2 — Feature Lineage	10
3 — Model Lineage	13
4 — Prompt Lineage	16
5 — Agent Lineage	19
6 — Knowledge Lineage	22
Platform Comparison	25
Impact Analysis	29
Root Cause Analysis	31
Compliance Reporting	33
AI Traceability & Explainability	36
Lineage for Multi-Agent AI Systems	39

Executive Summary

Key Finding: Lineage has expanded from a single technical concern (which tables feed which other tables) into a six-layer graph spanning raw data, computed features, trained models, prompts and retrieved context, autonomous agent decisions, and derived knowledge structures. Most enterprises have mature technical (table-level) lineage but significant — often complete — gaps in the five AI-era layers. These gaps are not cosmetic: they are precisely where root-cause analysis of AI failures, compliance evidence for AI regulation, and explainability for autonomous agent decisions must occur, and where they currently cannot.

This report examines lineage as an end-to-end discipline spanning six distinct but interconnected layers: data lineage (the established foundation), feature lineage (the ML-era extension), model lineage (training-to-artifact traceability), prompt lineage (the generative AI extension), agent lineage (the emerging frontier for autonomous systems), and knowledge lineage (for graph-structured enterprise knowledge). For each layer, the report examines what changed from the prior layer, what existing tooling covers versus what remains a gap, and how the major lineage platforms — OpenLineage, Marquez, DataHub, OpenMetadata, and Collibra, alongside emerging entrants — address (or don't yet address) each layer.

The report then examines four cross-cutting applications of lineage — impact analysis, root cause analysis, compliance reporting, and AI traceability/explainability — each of which depends on different combinations of the six layers, and each of which is correspondingly only as complete as the weakest layer it depends on. The report closes by examining how lineage itself must evolve for multi-agent AI systems, where a single user request may fan out into dozens of agent-to-agent and agent-to-tool interactions, each a potential lineage edge that current platforms were not designed to capture.

Six Critical Findings

1. Lineage Completeness Decreases Monotonically Across the Six Layers

Data lineage (Layer 1) is mature at most enterprises with established data platforms — OpenLineage and catalog-integrated lineage cover the majority of transformation pipelines. Feature lineage (Layer 2) is moderately covered where feature stores are adopted, but adoption itself is partial. Model lineage (Layer 3) is generally well-covered within a single ML platform but fragments across multi-platform estates. Prompt lineage (Layer 4), agent lineage (Layer 5), and knowledge lineage (Layer 6) are, for most organizations as of 2026, either absent or implemented as custom, non-standardized logging — no enterprise examined in this research has end-to-end coverage across all six layers with a single integrated lineage graph.

2. The 'Lineage Gap' Is Where AI Incidents Become Unexplainable

When an AI system produces an incorrect or harmful output, the investigation path runs backward through the layers: agent decision → prompt and retrieved context → model version → features → source data. A break in this chain at any layer halts the investigation. Because layers 4-6 are the least mature, most AI incident investigations in 2026 terminate at 'the model produced this output' without being able to establish why — the most common single root cause cited in this report's incident analysis section is not a data quality issue or a model issue per se, but a lineage gap that prevented determining which of several plausible causes was actually responsible.

3. OpenLineage Has Become the De Facto Interchange Standard, But Coverage Above Layer 2 Is Inconsistent

OpenLineage's job/run/dataset model and facet-based extensibility have made it the most widely-adopted open standard for data and (with custom facets) feature lineage. However, the specification's core abstractions (jobs producing/consuming datasets) map awkwardly onto model training runs, prompt executions, and agent tool calls — extensions exist but are implemented inconsistently across the ecosystem, meaning OpenLineage-based lineage graphs frequently have rich Layer 1-2 coverage and sparse, vendor-specific Layer 3+ coverage.

4. Catalog Platforms Are Converging on Graph-Native Lineage Models, Which Favors AI-Era Extension

DataHub and OpenMetadata's graph-based metadata models — where lineage is represented as edges in a property graph rather than a separate lineage-specific data structure — provide a more natural extension point for new node/edge types (prompts, agent runs, knowledge graph entities) than earlier relational-metadata catalog architectures. This architectural choice, made before the AI-era lineage requirements were fully understood, has turned out to be advantageous for the extensions now being built.

5. Compliance Frameworks Are Driving Lineage Investment Faster Than Internal Operational Needs

While the operational case for AI/prompt/agent lineage (faster incident resolution, better debugging) is strong, the EU AI Act's documentation requirements for high-risk systems — requiring demonstrable training data provenance and decision traceability — are emerging as the more urgent forcing function for enterprises to invest in Layers 3-5, particularly for organizations with EU exposure facing phased enforcement through 2027.

6. Multi-Agent Systems Will Require a Lineage Model Extension That Doesn't Yet Exist in Mature Form

Single-agent lineage (Layer 5) can, with effort, be represented as an extended trace — a linear-ish (if branching) sequence of tool calls and decisions. Multi-agent systems, where independent agents communicate via protocols like A2A and may operate concurrently with overlapping context, require a lineage model that captures not just sequences but agent-to-agent communication graphs, shared-memory read/write conflicts, and emergent coordination patterns — a fundamentally graph-shaped (not trace-shaped) lineage problem that current distributed tracing-derived approaches (OpenTelemetry-based agent observability) are not natively designed to represent.

The Six-Layer Lineage Model

The table below provides a navigational overview of the six lineage layers examined in this report, the question each layer answers, and its typical maturity level across enterprises as of 2026. Detailed analysis of each layer follows in the numbered sections.

#	Layer	Core Question Answered	Typical Maturity (2026)	Primary Tooling
1	Data Lineage	Which source columns/tables, through which jobs, produced this table/column?	High — mature and widely adopted	OpenLineage, dbt, Spline, catalog-native
2	Feature Lineage	Which raw data and transformations produced this feature, and which models consume it?	Medium — strong where feature stores adopted, absent elsewhere	Tecton, Hopsworks, Feast (limited), catalog extensions
3	Model Lineage	Which data, features, code version, and hyperparameters produced this trained model artifact?	Medium-High within single ML platforms; fragments across multi-platform estates	MLflow, SageMaker Lineage, Vertex ML Metadata
4	Prompt Lineage	Which prompt template version, combined with which retrieved context, produced this LLM output?	Low — mostly custom logging, inconsistent standards	Langfuse, Helicone, custom instrumentation
5	Agent Lineage	Which sequence of tool calls, decisions, and data sources led to this agent action?	Low — emerging, largely custom on OpenTelemetry	OpenInference/OpenTelemetry-based, framework-native traces
6	Knowledge Lineage	Which source documents and extraction runs produced this graph entity/relationship?	Low — niche outside GraphRAG-specific implementations	Custom extraction pipeline logging, emerging catalog extensions

Table 1: The Six-Layer Lineage Model

Why Six Layers, and Why This Order

The six layers are ordered to reflect both historical emergence and dependency structure: each layer depends on the layers before it for full traceability, while introducing lineage concerns that didn't exist in prior layers. Data lineage (Layer 1) is the foundation — without knowing where data comes from, no higher layer's lineage can be complete, since features, models, prompts, agents, and knowledge graphs are all ultimately derived from data. Feature lineage (Layer 2) extends data lineage with the specific transformation and point-in-time semantics that ML requires. Model lineage (Layer 3) extends feature lineage with training-run provenance — the same feature set can produce different models depending on code version, hyperparameters, and random seeds. Prompt lineage (Layer 4) is the first layer where the 'data' being traced (retrieved context, prompt templates) may not have existed as structured data in the traditional sense — introducing fundamentally different traceability requirements. Agent lineage (Layer 5) extends prompt lineage across multiple sequential (or concurrent) LLM interactions and tool calls that constitute a single agent task. Knowledge lineage (Layer 6) is somewhat orthogonal — it traces how source documents become graph-structured knowledge, which then becomes an input to layers 4-5 (retrieved/traversed by agents) — making it both a foundation for and a consumer of the lineage chain.

How to Read This Report: Each layer section follows a consistent structure: What Changed From the Prior Layer, What Existing Tooling Covers, What Remains a Gap, and Platform Capability Notes. This structure is designed to make explicit exactly where the lineage chain breaks for a given organization — readers should expect to identify their own organization's lineage maturity layer-by-layer rather than receiving a single 'mature' or 'immature' verdict.

1

Data Lineage

Column- and table-level transformation lineage — the established foundation

What This Layer Answers

Data lineage answers: for a given table or column, which source tables/columns, processed through which jobs/queries/transformations, produced it — and conversely, for a given source table, which downstream tables/columns depend on it. This is bidirectional by necessity: backward lineage supports root cause analysis ('why does this number look wrong'), while forward lineage supports impact analysis ('what breaks if I change this').

What Existing Tooling Covers

- **Job/run/dataset capture:** OpenLineage's core model — every pipeline run emits events describing which datasets it read and wrote, with column-level detail via facets — is widely integrated into Spark, dbt, Airflow, Flink, and other major processing engines
- **Transformation-as-code lineage:** dbt's compiled DAG provides exact, deterministic lineage for SQL-based transformations, since the transformation logic itself is the lineage source of truth — no separate capture mechanism is needed beyond parsing the dbt project
- **Column-level lineage via SQL/Spark parsing:** tools like Spline (for Spark) and SQL-parsing-based lineage extractors can derive column-level lineage automatically from query plans without requiring manual annotation
- **Catalog-integrated lineage visualization:** DataHub, OpenMetadata, and Collibra all provide lineage graph visualization integrated with their broader catalog — allowing a user to navigate from a table's documentation directly to its upstream/downstream lineage

What Remains a Gap

- **Out-of-band/manual interventions:** as discussed in the companion Operational Excellence report's Production Failure Analysis, manual corrections (ad-hoc UPDATE statements, manually uploaded corrected files) are invisible to lineage tooling that only tracks pipeline-driven transformations — this remains the most common source of 'unexplained' data lineage gaps even in otherwise-mature implementations
- **Cross-platform lineage continuity:** when data moves between fundamentally different systems (e.g., from an operational database via CDC into a lakehouse, then via reverse ETL into a SaaS CRM), lineage capture at each system boundary depends on each system's individual OpenLineage/catalog integration — gaps at any boundary break the end-to-end chain, and CDC tools historically have had less mature lineage integration than transformation tools like dbt
- **Non-tabular and streaming lineage:** lineage for streaming pipelines (Kafka topics, stream processing jobs) is less uniformly captured than batch lineage — partly because streaming jobs are long-running rather than discrete runs, which doesn't map as cleanly onto OpenLineage's run-based model
- **Business/semantic lineage connection:** technical (column-level) lineage rarely connects automatically to business lineage (which KPIs/metrics are affected) — this connection requires the semantic layer (discussed in the companion Architecture Evolution report) to be both adopted and itself lineage-instrumented, which is inconsistent across organizations

Platform Capability Notes

OpenLineage

The specification itself, not a platform — defines the job/run/dataset/facet model that other tools implement. Integrations exist for Spark, Flink, dbt, Airflow, Dagster, and others. Column-level lineage is supported via the ColumnLineageDatasetFacet, though adoption of this specific facet varies by integration maturity.

Marquez

The reference implementation/metadata service for OpenLineage — stores and visualizes the lineage graph that OpenLineage events describe. Strong for organizations wanting a lightweight, purpose-built lineage store without adopting a full catalog platform, but lacks the broader governance/documentation features of DataHub, OpenMetadata, or Collibra.

DataHub

Graph-native metadata model where lineage is represented as edges between dataset/job/etc. entities in a property graph — supports both push-based (via OpenLineage or native emitters) and pull-based (via crawlers/connectors that parse query history) lineage ingestion. Column-level lineage is well-supported for major engines.

OpenMetadata

Similar graph-based approach to DataHub, with growing connector coverage for automated lineage extraction. Notable for integrating lineage directly into its data quality and observability features — a lineage edge can be annotated with the quality status of the upstream dataset.

Collibra

Enterprise-grade lineage with the deepest source-system connector library among catalog platforms, reflecting its long history in regulated-industry data governance. Lineage is tightly integrated with Collibra's policy and classification framework — a lineage edge can trigger policy propagation (e.g., a classification tag flows downstream automatically).

2

Feature Lineage

Extending data lineage with point-in-time semantics and cross-model blast radius

What Changed From Data Lineage

Feature lineage extends data lineage with two requirements that table-level lineage doesn't address: point-in-time correctness (a feature's lineage must capture not just 'what data produced this feature' but 'as of what timestamp', since the same feature definition produces different values at different points in time) and many-to-many consumption (a single feature is typically consumed by many models, and a single model consumes many features — making 'blast radius' analysis for feature changes a many-to-many graph traversal rather than the more tree-like structure of typical table lineage).

What Existing Tooling Covers

- **Feature definition versioning:** feature stores with native versioning (Tecton, Hopsworks) track which version of a feature's transformation logic was active at any point in time, supporting reproducibility of historical feature values
- **Feature-to-model consumption mapping:** mature feature stores maintain a registry of which models/training runs consumed which feature versions — answering 'if I change this feature's logic, which models are affected' (forward/impact direction)
- **Feature-to-source lineage:** feature definitions reference their underlying data sources (typically lakehouse tables), and this reference can be combined with Layer 1 data lineage to produce an end-to-end view from raw source table to served feature value
- **Point-in-time join lineage:** platforms with native point-in-time join support record the timestamp semantics used for each training dataset generation, supporting the question 'was this training dataset generated with correct point-in-time joins, or is there leakage'

What Remains a Gap

- **Feature lineage outside adopted feature stores:** the majority of feature lineage tooling is feature-store-native — organizations computing 'features' as ad-hoc SQL/dataframe transformations outside a feature store (common for long-tail models, as discussed in the companion Architecture Evolution report) have no feature lineage at all for those features, only whatever table-level lineage (Layer 1) happens to cover the underlying transformation
- **Cross-feature-store lineage fragmentation:** organizations using multiple feature stores (e.g., a cloud-native feature store for some teams, self-hosted Feast for others) have feature lineage siloed within each platform, with no unified view of 'all features derived from this source table' across platforms
- **Online/offline synchronization lineage:** lineage typically describes the offline (batch) computation path well, but the online serving path — including synchronization lag between offline and online stores — is less consistently captured, even though synchronization lag is a documented production failure mode (companion Operational Excellence report)
- **Feature lineage for 'context store' / agentic features:** as discussed in the companion Architecture Evolution report, the extension of feature-store concepts toward agent context (retrieval results, conversation summaries) is ad-hoc per organization — there is no standard feature lineage model for these emerging feature types

Platform Capability Notes

OpenLineage

No first-class feature or feature-store abstraction in the core specification. Organizations integrating feature store lineage with OpenLineage-based pipeline lineage typically do so via custom facets representing feature definitions as a dataset type — functional but non-standardized across implementations.

Marquez

Inherits OpenLineage's lack of native feature abstraction; feature lineage would need to be represented through the same custom-facet approach, with no out-of-box feature-store integration.

DataHub

Has explicit support for ML entity types (MLFeatureTable, MLFeature, MLModel) in its metadata model, allowing feature definitions, their source lineage, and their consumption by models to be represented as first-class graph entities — among the stronger options for representing feature lineage within a general-purpose catalog.

OpenMetadata

Growing support for ML entity types, with feature lineage representable similarly to DataHub's approach, though with a smaller set of feature-store-specific connectors as of this writing — coverage depends on whether the specific feature store platform in use has an OpenMetadata connector.

Collibra

Feature lineage is generally handled via Collibra's AI Governance module rather than the base lineage capability — integrations with major feature store platforms exist but depth varies; Collibra's strength remains the policy propagation aspect (a feature derived from classified data inherits that classification).

Emerging: Feature-store-native lineage (Tecton, Hopsworks)

The most complete feature lineage as of 2026 remains within feature-store-native lineage graphs rather than general-purpose catalogs — Tecton and Hopsworks both provide feature-to-source and feature-to-model lineage natively, but this lineage is generally not exported to or unified with broader enterprise catalog lineage without custom integration work.

3

Model Lineage

Training-run provenance — code, data, hyperparameters, and artifact versioning

What Changed From Feature Lineage

Model lineage extends feature lineage with the additional dimensions that determine a trained model artifact's behavior beyond its input features: the exact code version (training script, preprocessing logic), hyperparameters, random seeds, compute environment (library versions, hardware), and the specific snapshot of training data (which, per Layer 2, requires point-in-time feature values). Two training runs using identical features can produce meaningfully different models if any of these additional dimensions differ — model lineage exists to make this difference traceable rather than an unrecorded variable.

What Existing Tooling Covers

- **Experiment tracking as lineage substrate:** MLflow, Weights & Biases, and similar experiment tracking tools capture code version (often via git commit hash), hyperparameters, metrics, and artifact references for each training run — providing the raw lineage data even when not framed explicitly as 'lineage'
- **Model registry versioning:** MLflow Model Registry, SageMaker Model Registry, and Vertex Model Registry maintain version history for registered models, linking each version back to its producing training run and forward to its deployment history (which endpoints/environments serve this version)
- **Training data snapshot references:** platforms with integrated feature store + training + registry (Databricks, SageMaker, Vertex AI) can reference the exact feature data snapshot (often via lakehouse time-travel, Section 3 of the companion Architecture Evolution report) used for a given training run, providing data-to-model traceability within that platform
- **Deployment lineage:** tracking which model version is/was deployed to which serving endpoint at which time — supporting both rollback ('redploy the previous version') and historical investigation ('which model version produced this prediction on this date')

What Remains a Gap

- **Cross-platform model lineage fragmentation:** an organization using one platform for training (e.g., a Databricks notebook with MLflow) and a different platform for serving (e.g., a custom Kubernetes-based serving infrastructure) has model lineage that's complete within MLflow but doesn't automatically connect to production serving logs without custom integration — meaning 'which model version produced this specific production prediction six months ago' can require manual correlation across systems
- **Foundation model / third-party LLM lineage:** for models that are fine-tuned or used as-is from a third-party provider (OpenAI, Anthropic, etc.), 'model lineage' in the traditional training-run sense doesn't apply in the same way — the relevant lineage becomes which model version/snapshot (often identified by a model API version string controlled by the provider, not the enterprise) was used, which is a fundamentally different and less-enterprise-controlled lineage record
- **Ensemble and pipeline model lineage:** when a production system's 'model' is actually a pipeline of multiple models (e.g., a retrieval model, a re-ranking model, and a generation model), lineage tooling designed around single-model training runs doesn't naturally represent the composite lineage of the overall pipeline's behavior
- **Continuous/online learning lineage:** for models that update continuously (online learning) rather than via discrete training runs, the experiment-tracking-based lineage model (one run = one lineage record) doesn't

map cleanly — lineage for continuously-updating models remains an edge case most tooling doesn't address well

Platform Capability Notes

OpenLineage

No native model/experiment abstraction — the specification's job/run/dataset model can represent a training run as a 'job' with the trained model as an output 'dataset', but this is a loose fit; most organizations use OpenLineage for the data-to-feature portion of model lineage (Layers 1-2) and a dedicated ML platform (MLflow etc.) for the training-to-model portion, with limited integration between the two lineage representations.

Marquez

Same limitation as OpenLineage given its role as the reference implementation — model lineage specifically is outside its primary scope.

DataHub

First-class MLModel, MLModelGroup, and related entity types allow model versions, their training run lineage, and their deployment lineage to be represented in the same graph as data and feature lineage — DataHub's MLflow integration can ingest experiment tracking data into this unified graph, providing one of the more complete cross-layer (1-3) lineage views among general-purpose catalogs.

OpenMetadata

Similarly growing ML entity support; integration depth with major experiment tracking/registry platforms (MLflow, SageMaker) is actively developing, with coverage varying by specific platform combination.

Collibra

Model lineage is addressed through the AI Governance module, which focuses more on model risk/compliance metadata (approval status, risk tier, documentation completeness) than on granular training-run provenance — Collibra is typically positioned as the governance layer consuming lineage from MLflow/registry platforms rather than as the lineage source of truth for training provenance itself.

Emerging: MLflow / SageMaker Lineage / Vertex ML Metadata as de facto standards

Within their respective platforms, these provide the most complete model lineage available — the gap is exclusively at the cross-platform level. Emerging open standards efforts (extending OpenLineage's facet model to better represent ML experiment/model entities) are in early stages as of 2026 but not yet broadly adopted.

4

Prompt Lineage

The first layer where 'lineage' means tracing non-deterministic, context-assembled inputs

What Changed From Model Lineage

Prompt lineage represents a qualitative shift from the prior three layers. Layers 1-3 trace deterministic transformations of structured data through versioned code — given the same inputs and code version, the output is reproducible. Prompt lineage must trace: the prompt template version (itself versioned code, similar to Layer 1-3 artifacts), the retrieved context (the output of a vector/graph retrieval operation that depends on the current state of an index — Layers 1, 6, and the companion Architecture Evolution report's Section 9), and the LLM's output (non-deterministic, dependent on model version/provider state that the enterprise often doesn't control). Prompt lineage is therefore lineage for a process that is only partially reproducible even with complete lineage records — a fundamentally different guarantee than Layers 1-3 provide.

What Existing Tooling Covers

- **Prompt template versioning:** Langfuse and similar platforms provide git-like versioning for prompt templates, allowing 'which version of this prompt template was active when this response was generated' to be answered — this is the most mature aspect of prompt lineage as of 2026
- **Trace-level logging of inputs/outputs:** LLM observability platforms (Langfuse, Helicone, Arize Phoenix) log the full assembled prompt (including retrieved context inserted into the template) and the resulting output for each request — providing a record of what was actually sent to the model, even if not always structured as formal 'lineage'
- **Retrieval result logging:** the same platforms typically log which documents/chunks were retrieved for a given request, including similarity scores — allowing the question 'which source documents informed this response' to be answered, assuming the retrieval logging is correctly correlated with the corresponding generation request
- **Model version/provider metadata:** requests to LLM APIs typically include (or the response includes) the specific model version used — providing the 'which model snapshot generated this' record discussed as a Layer 3 gap for third-party models

What Remains a Gap

- **Connecting prompt lineage to upstream data/knowledge lineage:** the most significant gap. LLM observability platforms log 'these document chunks were retrieved' but typically don't connect this to Layer 1 (which source table/document version produced this chunk) or Layer 6 (which extraction run produced this knowledge graph entity that informed the response) — meaning the chain from 'this output was wrong' back to 'because this source document was outdated' frequently cannot be completed without manual cross-referencing between the LLM observability platform and the data catalog/lineage system, which are typically entirely separate tools with no integration
- **System prompt and configuration lineage:** system prompts (which encode business rules, safety constraints, and behavioral instructions) are often managed separately from user-facing prompt templates, with less consistent versioning/lineage — a change to a system prompt that affects all requests using a given application is a significant lineage event that's frequently under-tracked relative to per-request prompt template versions

- **Non-reproducibility as an inherent lineage limitation:** even with complete prompt lineage records, re-running the same prompt template + retrieved context + model version may not reproduce the same output (LLM outputs have inherent variability, and provider-side model updates can change behavior for a 'fixed' model version string) — prompt lineage can establish 'what was sent and what version was used' but cannot guarantee 'reproducing this would give the same result', a fundamental limitation that lineage consumers (especially for compliance purposes) need to understand explicitly
- **Multi-turn conversation lineage:** for multi-turn conversations, each turn's prompt includes prior conversation history — prompt lineage for turn N technically includes the lineage of turns 1 through N-1 as part of its input, creating a lineage record that grows with conversation length and that most tooling doesn't represent efficiently (often re-logging the full growing context at each turn rather than representing it as an incremental lineage chain)

Platform Capability Notes

OpenLineage

No native prompt/LLM abstraction in the core specification as of this writing. The job/run/dataset model could in principle represent an LLM call as a job with the prompt and retrieved context as input 'datasets' and the response as an output 'dataset', but no widely-adopted convention for this exists — prompt lineage in practice is captured by LLM-observability-specific tools (below) operating independently of OpenLineage-based data lineage.

Marquez

Not applicable — outside scope as the OpenLineage reference implementation, with the same gap as OpenLineage itself.

DataHub

No first-class prompt/LLM trace entities as of 2026, though DataHub's extensible entity model could accommodate custom entity types for prompts/LLM calls — this would require custom development rather than being available out-of-box. DataHub's strength remains as the potential integration point connecting prompt-level lineage (from Langfuse/Phoenix) to data/knowledge lineage, if such integration were built.

OpenMetadata

Similar position to DataHub — extensible but without native prompt lineage entities; the integration opportunity (connecting LLM observability to data catalog lineage) is recognized but not yet a standard out-of-box capability.

Collibra

Prompt lineage is addressed, to the extent it is, through the AI Governance module's prompt governance capabilities (Part 16 of the companion Operational Excellence report) — focused more on prompt version approval/review workflows than on per-request retrieval/output lineage, which remains the domain of LLM observability platforms.

Emerging: Langfuse, Arize Phoenix, Helicone as the prompt lineage source of truth

These platforms provide the most complete prompt-level lineage available — prompt template versions, retrieved context, and outputs, often with OpenTelemetry-compatible tracing (Langfuse, Phoenix). The emerging direction (not yet standard) is using OpenTelemetry/OpenInference trace context as the connective layer between these platforms and data-catalog lineage — since both increasingly support OpenTelemetry-based instrumentation, a shared trace ID could in principle link a retrieval span to the data lineage of the retrieved content's source, but this linkage is not yet implemented as a standard pattern across the ecosystem.



Agent Lineage

Tracing the full decision chain of an autonomous agent across tools and steps

What Changed From Prompt Lineage

Prompt lineage (Layer 4) traces a single request-response interaction with an LLM. Agent lineage extends this across the full sequence of steps that constitute a single agent task: an agent may make multiple LLM calls, call multiple tools (each potentially itself a traditional ML model, a database query, an API call, or another agent), and make decisions about which tools to call based on intermediate results — all before producing a final action or response. Agent lineage must capture this entire chain as a structured trace: not just 'what was the final output' but 'what was the sequence of reasoning, tool calls, and intermediate results that led there', since any step in this chain could be the source of an eventual error.

What Existing Tooling Covers

- **Distributed tracing as the structural foundation:** OpenTelemetry-based distributed tracing, extended with OpenInference's GenAI-specific semantic conventions, provides the span/trace structure needed to represent an agent's sequence of LLM calls and tool invocations as a single trace tree — this is the most mature aspect of agent lineage, directly inheriting from Part 12 of the companion Operational Excellence report's platform observability discussion
- **Framework-native execution traces:** agent orchestration frameworks (LangGraph, CrewAI, AutoGen, OpenAI Agents SDK — discussed in the companion Operational Excellence report's Part 17) typically provide their own execution trace/history representation, capturing the sequence of nodes/agents/tools executed for a given run — though these traces are framework-specific in structure and not standardized across frameworks
- **Tool call input/output logging:** for each tool call within an agent's execution, both the input parameters and the returned result are typically logged by observability platforms (Langfuse, Phoenix) — providing the raw data needed to determine 'what information did the agent have when it made this decision'
- **Decision point identification:** for frameworks with explicit control flow (LangGraph's graph-based execution), the trace can identify specific decision points (which branch of the graph was taken, and based on what condition) — providing more structured lineage than frameworks where the LLM's reasoning entirely determines the execution path

What Remains a Gap

- **Connecting agent traces to upstream lineage layers:** as with prompt lineage, agent execution traces typically exist in an LLM-observability silo separate from data/feature/model/knowledge lineage systems — an agent trace shows 'the agent called the customer-lookup tool and received this data' but doesn't typically connect 'this data' back to the source table's Layer 1 lineage, the feature's Layer 2 lineage if it was a computed feature, or the knowledge graph entity's Layer 6 extraction lineage if it came from a graph traversal
- **Reasoning traceability vs. action traceability:** agent traces reliably capture what tools were called with what inputs/outputs (action traceability) but capturing why the agent chose to call a particular tool — the LLM's 'reasoning' that led to that decision — is less reliably captured, particularly for models/frameworks that don't expose explicit chain-of-thought, or where chain-of-thought is generated but not retained in logs due to volume/cost considerations (Part 12 of the companion report)

- **Cross-agent and cross-session continuity:** when an agent's task spans multiple sessions (e.g., a long-running task that's paused and resumed, or a task handed off between different agent instances), maintaining a continuous lineage trace across this discontinuity requires explicit trace context propagation that's inconsistently implemented — many implementations effectively restart tracing at session boundaries, fragmenting what should be a single logical lineage chain
- **Agent identity in lineage records:** as discussed in the companion Operational Excellence report's Part 14, agent identity (distinct from the human user or service account) is an emerging, non-standardized concept — agent lineage records frequently lack a stable agent identity field, making it difficult to answer 'across all tasks, what has this specific agent (vs. this application generally) done' as agent identity standards mature
- **Cost/token attribution as a lineage dimension:** while not strictly 'lineage' in the traceability sense, cost attribution (which agent, which step, consumed how many tokens/how much compute) shares the same trace-based data model as agent lineage and is frequently needed for the same investigations (e.g., 'this agent's cost-per-task spiked — which step changed') — but is often captured by separate cost-monitoring tooling rather than unified with lineage traces

Platform Capability Notes

OpenLineage

No native agent/tool-call abstraction. As with prompt lineage, agent lineage in practice is captured by OpenTelemetry/OpenInference-based tracing rather than OpenLineage's job/run/dataset model — the two standards currently operate in largely separate domains (OpenLineage for data pipeline lineage, OpenTelemetry-derived approaches for agent/LLM tracing) without a standardized bridge.

Marquez

Outside scope, as with prior AI-era layers.

DataHub

No native agent trace entities as of 2026. As with prompt lineage, DataHub's extensibility makes it a plausible future integration point for connecting agent traces to data lineage, but this requires custom development; no standard connector for agent observability platforms exists yet.

OpenMetadata

Similar position — extensible metadata model without native agent lineage entities or standard integrations with agent observability platforms as of this writing.

Collibra

Agent governance (Part 16 of the companion report) within Collibra's AI Governance module focuses on policy definition (what agents are permitted to do) rather than execution-level lineage (what a specific agent run actually did) — the two are complementary but distinct, and Collibra's current emphasis is the former.

Emerging: OpenInference + OpenTelemetry as the agent lineage substrate

The most active area of standards development relevant to agent lineage is the extension of OpenTelemetry's GenAI semantic conventions (via OpenInference and related efforts) to cover agent-specific span types (tool calls, agent handoffs, multi-step reasoning chains). Arize Phoenix and Langfuse both support these conventions and are the most mature platforms for agent-level trace visualization as of 2026. The gap — connecting these traces to Layers 1-3 and 6 — remains the primary open integration challenge, and is the focus of early-stage efforts (not yet standardized) to propagate trace context across the data-lineage/agent-observability boundary.

6

Knowledge Lineage

Tracing how source documents become graph-structured entities and relationships

What This Layer Answers, and Its Relationship to the Other Layers

Knowledge lineage answers: for a given knowledge graph entity or relationship (as discussed in Section 6 of the companion Architecture Evolution report), which source documents, processed through which extraction/entity-resolution pipeline run, produced it. Unlike Layers 2-5, which form a roughly linear dependency chain (data → features → models → prompts → agents), knowledge lineage is somewhat orthogonal: it traces the construction of the knowledge graph itself (an input process, similar in character to Layer 1's table lineage but for graph-structured rather than tabular targets), while the knowledge graph subsequently becomes an input that Layers 4-5 consume (via GraphRAG retrieval/traversal). Knowledge lineage is therefore both a 'Layer 1-like' lineage problem for the graph's construction and a prerequisite for completing Layers 4-5's lineage chains when graph retrieval is involved.

What Existing Tooling Covers

- **Extraction pipeline run logging:** custom-built GraphRAG implementations typically log, for each pipeline run, which source documents were processed and which entities/relationships were extracted or updated — providing the raw data for knowledge lineage even when not exposed as a formal 'lineage' feature
- **Entity provenance attributes:** some knowledge graph implementations attach provenance metadata directly to entities/relationships as graph properties (e.g., 'source_document_id', 'extraction_run_id', 'extraction_timestamp') — a lightweight form of knowledge lineage embedded in the graph itself rather than in a separate lineage system
- **Community/summary lineage in GraphRAG patterns:** for GraphRAG implementations using the community-detection-and-summarization pattern (Microsoft GraphRAG-style), the summaries themselves can in principle be traced back to the entities/relationships (and thus source documents) that contributed to each community — though this traceability is rarely surfaced as a queryable lineage feature in current implementations

What Remains a Gap

- **No standard knowledge lineage model:** unlike Layers 1-3, which have OpenLineage/MLflow-equivalent standards (however incomplete in cross-platform coverage), knowledge lineage has no widely-adopted standard model at all as of 2026 — every GraphRAG implementation examined in this research handles provenance, if at all, through custom, implementation-specific approaches
- **Entity resolution lineage:** when entity resolution merges or splits entities (e.g., determining that 'Acme Corp' and 'Acme Corporation' are the same entity, as discussed in the companion Architecture Evolution report's Section 6), the lineage of this resolution decision — what evidence supported the merge, and could it be reversed if found incorrect — is rarely tracked, even though entity resolution errors are identified as a primary knowledge graph failure mode
- **Graph update/deletion lineage:** when source documents are updated or deleted and the graph is correspondingly updated (removing or modifying entities/relationships), lineage of these update operations — what changed, when, and why — is typically not tracked with the same rigor as the initial extraction, creating a gap precisely where the 'stale knowledge served as current' failure mode (companion Operational Excellence report's Production Failure Analysis) originates

- **Connecting knowledge lineage to Layer 1:** source documents themselves often live in document repositories, content management systems, or unstructured data lakes that have their own (often weaker) Layer 1 lineage — connecting 'this graph entity came from this document' to 'this document came from this source system with this access control' requires bridging knowledge lineage with document-repository lineage, a connection that's rarely made explicit
- **Multi-hop retrieval lineage:** when an agent's response is informed by a multi-hop graph traversal (not just a single entity lookup), the lineage of the response should ideally trace through every hop of the traversal back to each contributing entity's source documents — current GraphRAG implementations generally log the final traversal result used in the prompt (connecting to Layer 4) but not the full traversal path with per-hop provenance

Platform Capability Notes

OpenLineage

No knowledge-graph-specific abstractions. In principle, an extraction pipeline run could be represented as an OpenLineage job with source documents as input datasets and extracted entities as output datasets, but graph-structured outputs (entities with relationships, as opposed to rows in a table) don't map naturally onto OpenLineage's dataset-as-table-like abstraction — no widely-adopted convention for this exists.

Marquez

Outside scope, as with other AI-era layers.

DataHub

DataHub's graph-native architecture is, architecturally, the most natural fit among general-purpose catalogs for representing knowledge graph entities and their provenance as part of the same metadata graph — but as of 2026 this requires custom entity type definitions; no out-of-box 'knowledge graph provenance' feature exists, and no major GraphRAG framework has a standard DataHub integration for provenance export.

OpenMetadata

Similar position to DataHub — graph-based architecture provides a plausible foundation, but knowledge lineage specifically is not a developed feature area as of this writing.

Collibra

No specific knowledge graph lineage capability identified; Collibra's strength in document/content classification could in principle extend to tracking which classified documents fed which knowledge graph construction processes, but this integration is not a standard capability.

Emerging: Custom provenance metadata within graph databases themselves

The most common current approach — embedding provenance as node/relationship properties within Neo4j, Stardog, or other graph databases (companion Architecture Evolution report, Section 6) directly, rather than in a separate lineage system — has the advantage of co-locating lineage with the data it describes, but the disadvantage of being queryable only within that graph database's query language, not through a general-purpose lineage/catalog interface. No emerging standard has yet achieved meaningful adoption for unifying knowledge lineage with the other five layers; this is identified in this report's findings as the least mature of all six lineage layers.



Platform Comparison

Six lineage layers, five platforms, and the emerging entrants between them

This section consolidates the platform capability notes from Sections 1-6 into a single comparative view, then examines operational characteristics (deployment model, query interface, extensibility) that affect how readily each platform can be extended to cover the AI-era layers where gaps remain.

Layer Coverage Matrix

Platform	L1: Data	L2: Feature	L3: Model	L4: Prompt	L5: Agent	L6: Knowledge
OpenLineage	Strong (spec + integrations)	Via custom facets only	Loose fit via job/dataset model	Not addressed	Not addressed	Not addressed
Marquez	Strong (reference impl.)	Via custom facets only	Outside scope	Not addressed	Not addressed	Not addressed
DataHub	Strong, graph-native	Native ML entity types	Native MLModel entities + MLflow integration	Extensible, no native entities	Extensible, no native entities	Architecturally suited, not implemented
OpenMetadata	Strong, growing connectors	Growing ML entity support	Growing, platform-dependent	Extensible, no native entities	Extensible, no native entities	Architecturally suited, not implemented
Collibra	Very strong, enterprise connectors	Via AI Governance module	Via AI Governance module (risk-focused)	Via AI Governance (prompt governance)	Via AI Governance (policy-focused)	Not a developed capability

Table 2: Lineage Layer Coverage by Platform

Operational Characteristics

Platform	Deployment Model	Query Interface	Extensibility Model	Best Fit
OpenLineage	Specification — implemented within producing tools (Spark, dbt, Airflow, etc.)	Via consuming backend (Marquez, DataHub, etc.)	Facet-based — new facet types extend event schema	Standardizing how lineage events are emitted across heterogeneous pipeline tools
Marquez	Self-hosted service (API + UI + storage backend)	REST API + web UI lineage graph	Limited — primarily an OpenLineage event store/visualizer	Lightweight, OpenLineage-native lineage store without broader catalog overhead

Platform	Deployment Model	Query Interface	Extensibility Model	Best Fit
DataHub	Self-hosted (OSS) or DataHub Cloud (managed)	GraphQL API + web UI + Python/Java SDKs	High — custom entity/aspect types via metadata model extensions	Engineering-led organizations wanting a unified, extensible metadata graph spanning data through ML entities
OpenMetadata	Self-hosted (OSS) or Collate (managed)	REST API + web UI + SDKs	High — custom entity types, growing connector ecosystem	Organizations wanting DataHub-like extensibility with a different connector/ingestion architecture
Collibra	SaaS (Collibra Cloud) primarily, with on-prem options	Web UI-centric, REST API for integration	Moderate — configuration-driven via Collibra's metamodel, less code-extensible than DataHub/OpenMetadata	Regulated enterprises prioritizing policy/governance integration over engineering-led custom extension

Table 3: Platform Operational Characteristics

Emerging Platforms and Standards

OpenInference (Arize)

An open standard (built on OpenTelemetry semantic conventions) for instrumenting LLM applications — defining span attributes for prompts, completions, embeddings, retrievals, and (increasingly) agent/tool-call spans. Functions as the de facto standard for Layer 4-5 trace structure, analogous to OpenLineage's role for Layer 1. Adopted by Arize Phoenix natively and supported by Langfuse and other observability platforms — the most likely foundation for future Layer 4-5 lineage standardization, though it remains focused on trace structure rather than lineage graph semantics (connecting traces to upstream data/knowledge entities).

MLflow Tracing

MLflow's tracing capability (extending its established experiment-tracking role into LLM/agent observability) represents an attempt to unify Layer 3 (model lineage, MLflow's original domain) with Layer 4-5 (prompt/agent tracing) within a single platform — relevant because it's one of the few efforts explicitly attempting to bridge these layers rather than treating them as separate tooling domains, though adoption for the Layer 4-5 portion specifically is earlier-stage than MLflow's established Layer 3 usage.

Unity Catalog Lineage (Databricks) / Microsoft Purview

Platform-native lineage within major lakehouse platforms (companion Architecture Evolution report, Section 3) increasingly extends across Layers 1-3 within the platform's boundary, with Databricks' Mosaic AI and Microsoft's Copilot/Purview integrations beginning to extend toward Layer 4-5 for AI workloads run on those platforms. The relevance to this report: as discussed in the companion Architecture Evolution report's Future Forecast, platform-native lineage extension may outpace general-purpose catalog extension for organizations consolidated on a single major platform, though this creates platform lock-in for the lineage graph itself.

W3C PROV and Linked Data Provenance Standards

Older (pre-AI-era) provenance standards from the semantic web tradition (companion Architecture Evolution report, Section 6) provide a formally-specified, general-purpose model for representing 'what produced this, from what, using what process' — theoretically applicable across all six lineage layers including knowledge lineage (Layer 6), where it originated. Adoption for AI-era lineage remains minimal as of 2026 — the AI/ML lineage ecosystem has largely developed independently of semantic web provenance standards despite conceptual overlap, though some research-oriented implementations reference PROV for knowledge graph provenance specifically.

Platform Selection Implication: No single platform in this comparison provides mature coverage across all six layers. Organizations seeking the broadest current coverage with the most extensibility for the immature layers should weight DataHub or OpenMetadata's graph-native, extensible metadata models — but should expect to build custom integration work for Layers 4-6 regardless of platform choice. Organizations in regulated industries where Collibra's policy/governance integration is already central to their compliance posture face a tradeoff: stronger governance-process integration for the layers Collibra covers, against weaker native extensibility for the layers it doesn't yet cover.

Impact Analysis

Impact analysis answers the forward-direction lineage question: given a proposed or actual change to a data source, feature, model, prompt, or knowledge graph component, what downstream assets, models, agents, or decisions are affected? This is the primary use case for lineage during change management — before modifying a table schema, retiring a feature, updating a model, or changing a knowledge graph ontology, impact analysis identifies the blast radius.

Impact Analysis by Layer

Layer 1 → Downstream: Schema/Table Changes

The most mature impact analysis use case — given a proposed column rename, type change, or table deprecation, lineage tooling can enumerate every downstream table, dashboard, and (where Layer 2 lineage connects) feature that depends on it. DataHub, OpenMetadata, and Collibra all provide this as a core feature, typically with the ability to filter the impact set by criticality or ownership for prioritized communication to affected teams.

Layer 2 → Downstream: Feature Definition Changes

Where feature stores provide feature-to-model lineage (Section 2), impact analysis can answer 'if I change this feature's transformation logic, which models will receive different values and need re-evaluation' — but this analysis is confined to features registered in the feature store; ad-hoc features computed outside feature stores have no corresponding impact analysis, meaning a change to shared upstream logic (e.g., a common SQL transformation reused across ad-hoc feature computations) requires manual identification of affected models.

Layer 3 → Downstream: Model Version Changes

Impact analysis for model updates is bounded by deployment lineage (Section 3) — which serving endpoints, applications, and (where Layer 5 connects) agents consume a given model version. For models used as tools within agent systems, the impact of a model update propagates to every agent task that might invoke that tool — a connection that requires Layer 3-5 integration that, per Section 5, is generally not implemented.

Layer 4 → Downstream: Prompt Template Changes

Impact analysis for prompt template changes is typically scoped within a single application (which application uses this template) rather than across the broader lineage graph — Langfuse and similar platforms can show 'this template is used by these applications' but generally cannot show 'this template change affects responses that inform these downstream agent decisions or knowledge graph updates', since that would require Layer 4-5-6 integration.

Layer 5 → Downstream: Agent Tool/Configuration Changes

Impact analysis for changes to an agent's available tools, permissions, or orchestration logic is largely framework-dependent — LangGraph's graph definition makes the set of possible execution paths explicit (impact analysis can reason about which paths a change affects), while less-structured agent frameworks (where the LLM's reasoning determines tool selection dynamically) make 'which scenarios would this change affect' inherently harder to enumerate statically — impact analysis in these cases often requires empirical evaluation (running test scenarios) rather than lineage-graph traversal.

Layer 6 → Downstream: Ontology/Schema Changes

Impact analysis for knowledge graph ontology changes (adding/modifying entity or relationship types) requires identifying which existing entities/relationships would need re-extraction or migration under the new ontology, and which downstream GraphRAG queries (and thus, transitively, which agent behaviors) depend on the affected graph structure — per Section 6, this requires knowledge lineage that's generally not implemented, making ontology changes in practice a high-risk, manually-assessed change category.

Practical Implication: Impact analysis completeness mirrors the lineage completeness gradient described in this report's Executive Summary — changes to Layer 1 assets have the most reliable impact analysis, while changes to Layer 4-6 assets (prompt templates, agent configurations, knowledge graph ontologies) frequently must be assessed through testing/evaluation rather than lineage-graph traversal, because the lineage graph itself doesn't extend reliably into these layers. Organizations should treat changes to AI-era assets with correspondingly more conservative change management (staged rollout, evaluation-based validation) precisely because lineage-based impact analysis cannot yet provide the same confidence it does for Layer 1 changes.

Root Cause Analysis

Root cause analysis is the backward-direction counterpart to impact analysis: given an observed problem (an incorrect dashboard number, a model performance regression, an AI hallucination, an agent taking an inappropriate action), trace backward through the lineage graph to identify the originating cause. Root cause analysis is where lineage gaps are most directly costly — an investigation that hits a gap doesn't just produce an incomplete report, it can leave the actual root cause unaddressed, allowing recurrence.

The Root Cause Investigation Chain for AI Incidents

For an AI system incident (e.g., an agent provided incorrect information to a customer), the investigation chain typically proceeds backward through the layers in this order, with each step's success depending on the corresponding layer's lineage maturity:

Step 1 — Agent Lineage (Layer 5)

Was the agent's action/response consistent with its tool call results and reasoning trace? If the trace shows the agent correctly synthesized the information it received, the root cause lies upstream (Step 2); if the trace shows the agent misinterpreted correct information, the root cause may be a model/prompt issue (Step 2 or 3) rather than a data issue.

Step 2 — Prompt Lineage (Layer 4)

What context was retrieved and included in the prompt that produced the agent's response? If the retrieved context itself was incorrect or outdated, the root cause lies further upstream (Step 3 or 4); if the retrieved context was correct but the model's synthesis of it was wrong, the root cause may be a model limitation (a Layer 3 concern, but one less amenable to lineage-based root cause — more a model evaluation/selection concern).

Step 3 — Knowledge Lineage (Layer 6, if GraphRAG involved)

If the retrieved context came from a knowledge graph traversal, was the traversed entity/relationship data correct and current? An entity resolution error or stale graph data (per Section 6's failure modes) would be identified here — but per Section 6, this step is the least likely to be completable due to minimal knowledge lineage tooling.

Step 4 — Data/Document Lineage (Layer 1)

If the retrieved context (directly, or via the knowledge graph) traces back to a source document or table, was that source correct and current at the time of retrieval? This is where the investigation connects to the most mature lineage layer — but only if Steps 2-3 successfully connected the prompt/knowledge lineage to this layer, which per Sections 4 and 6 is frequently not the case.

Step 5 — Feature/Model Lineage (Layers 2-3, if a traditional ML component is involved)

If the agent's task involved a traditional ML model (e.g., a risk score used as one input to the agent's decision), was that model's prediction based on correct, current feature values? This branch of the investigation follows the more mature Layer 2-3 lineage, but only if Layer 5's tool-call logging correctly identifies which model was called and with what inputs — connecting Layer 5 to Layer 3 (Section 5's gap).

Common Root Cause Analysis Failure Patterns

- **Investigation terminates at 'the model said X':** when Steps 2-4 cannot be completed due to missing prompt-to-data lineage connections, the investigation's conclusion is effectively 'the model produced this output given this input' without being able to determine why that input led to that output, or whether the input itself was correct — frequently misattributed to 'model hallucination' when the actual root cause was upstream data/knowledge issues (as discussed in the companion Operational Excellence report's Production Failure Analysis)
- **Multiple plausible root causes, none confirmable:** without complete lineage, an investigation may identify 2-3 plausible explanations (stale retrieved document, model limitation, agent reasoning error) without being able to determine which actually occurred — leading to remediation efforts that address a guessed-at cause, which may or may not be the actual one, with the issue potentially recurring if the guess was wrong
- **Time-to-resolution inflation:** each lineage gap requires manual cross-referencing between systems (e.g., manually checking the data catalog for a source document's update history after the LLM observability platform identified it as a retrieved source) — multiplying investigation time relative to a fully-connected lineage graph where this would be a single traversal query
- **Survivorship bias in incident reporting:** incidents where root cause analysis successfully completes (typically Layer 1-3-dominant issues, where lineage is mature) are over-represented in incident postmortems relative to incidents where investigation stalled at a Layer 4-6 gap and was closed as 'inconclusive' — creating a skewed organizational understanding of where AI incidents actually originate

Recommendation: Organizations should explicitly track 'investigation completeness' as a metric for AI-related incidents — what fraction of incidents reach a confirmed root cause vs. terminate at a lineage gap — both as a direct measure of operational risk and as a prioritization input for where to invest in closing lineage gaps (Sections 4-6). An incident category that frequently terminates at 'retrieved context couldn't be traced to its source document's current state' is a strong, evidence-based case for prioritizing the Layer 4-to-Layer 1 lineage connection over other candidate investments.

Compliance Reporting

Lineage is the structural backbone of compliance evidence for the data and AI regulations discussed in the companion Operational Excellence report's Part 15. This section maps specific compliance requirements to the lineage layers that must support them, highlighting where current lineage maturity (or immaturity) directly determines compliance readiness.

Regulation/Framework	Lineage Requirement	Primary Layers	Current Readiness
GDPR / CCPA (data subject rights)	Trace all locations/derivatives of an individual's data for erasure/portability requests, including derived features, embeddings, and graph entities	L1, L2, L6	Moderate for L1-2 (lakehouse delete propagation); weak for L6 (vector/graph derivatives often untracked)
GDPR Article 30 (records of processing)	Document end-to-end data flows including AI processing purposes	L1-L4	Moderate — L1 flows documentable; L3-4 AI processing purposes often documented separately from technical lineage
EU AI Act (high-risk systems — training data documentation)	Demonstrate provenance of training data, including data quality and bias characteristics, for the trained model	L1, L2, L3	Moderate-High where feature stores + ML platforms provide L2-3 lineage; weak where ad-hoc feature computation bypasses this
EU AI Act (high-risk systems — post-market monitoring)	Continuous traceability from production AI decisions back to model version and (where applicable) input data/context	L3, L4, L5	Low — requires L3-5 connection that Section 5 identifies as largely unimplemented
ISO 42001 (AI management systems)	Documented AI system inventory with lineage to underlying data and models	L1-L3	Moderate — achievable with disciplined model registry + feature store practices, though cross-platform fragmentation (Section 3) remains a gap
NIST AI RMF (Measure/Manage functions)	Continuous monitoring data connected to risk management decisions, requiring traceability from observed issues to contributing factors	L1-L6 (incident-dependent)	Low-Moderate — depends entirely on root cause analysis completeness (this report's prior section), which varies by incident type
SOC 2 / ISO 27001 (access audit trails)	Demonstrate who/what accessed which data, including AI agents	L1, L5	Moderate for L1 (standard access logging); low for L5 (agent identity in access logs is non-standard, per Section 5)
Sector-specific (RBI/MAS — AI explainability for credit/financial decisions)	Trace a specific automated decision back to the input data and model version that produced it, in human-interpretable form	L1-L5	Low for AI/LLM-based decisions (full L1-5 chain rarely complete); Moderate for traditional ML credit models with mature L1-3 lineage

Table 4: Compliance Requirements Mapped to Lineage Layers

The Compliance Evidence Generation Problem

As discussed in the companion Operational Excellence report's Part 15, the recommended strategic approach to multi-framework compliance is a unified evidence-collection architecture rather than framework-by-framework point solutions. Lineage is the connective tissue of this architecture: most compliance evidence requirements (Table 4) are fundamentally lineage queries — 'show me everything derived from this data subject's records', 'show me the training data provenance for this model', 'show me the decision chain for this automated decision'. A unified evidence layer is, in large part, a unified lineage graph with compliance-specific query templates and export formats layered on top.

Where Lineage Gaps Create Compliance Exposure

- **Layer 6 gaps and erasure requests:** if an individual's data was used to construct knowledge graph entities/relationships (e.g., a customer entity with linked records), and the knowledge graph has no lineage connecting entities back to source records, an erasure request may successfully delete the source data while leaving derived graph structure intact and queryable — a GDPR compliance gap that's invisible until specifically audited, since the graph 'looks' fine and the deletion 'succeeded' from the source system's perspective
- **Layer 3-5 gaps and EU AI Act post-market monitoring:** the EU AI Act's post-market monitoring requirements for high-risk systems effectively require the Layer 3-5 connection that Section 5 identifies as the primary agent lineage gap — organizations deploying agentic systems in EU AI Act high-risk categories (a determination that itself requires careful assessment) face a compliance requirement that current tooling does not natively satisfy, requiring custom instrumentation
- **Layer 4 gaps and explainability for automated decisions:** sector-specific explainability requirements (RBI/MAS FEAT principles, discussed in the companion Operational Excellence report's Part 15) for AI-assisted financial decisions require connecting a specific decision to the specific information that informed it — for LLM-based decision support, this is a Layer 4 lineage requirement that, per Section 4, is the layer where connecting prompt lineage to upstream data lineage is the primary gap
- **Cross-layer audit trail continuity:** compliance audits often sample specific transactions/decisions and request the complete supporting evidence trail — a single broken link anywhere in the L1-L5 chain for a sampled item can result in an audit finding, even if the organization's lineage is largely complete elsewhere; audit risk is therefore disproportionately driven by the least-complete layer relevant to the sampled population, not the average completeness across all layers

Compliance Recommendation: Organizations preparing for EU AI Act enforcement phases (through 2027) or sector-specific AI explainability requirements should conduct a lineage gap assessment specifically scoped to their high-risk/regulated AI use cases — identifying, for each such use case, which of the six layers' lineage is required for that use case's compliance obligations, and which of those layers currently has gaps. This targeted assessment (rather than attempting comprehensive six-layer lineage everywhere) allows prioritized remediation focused on actual compliance exposure rather than theoretical completeness.

AI Traceability & Explainability

Traceability and explainability are related but distinct: traceability (the focus of Sections 1-6 and the Impact/Root Cause sections) is about reconstructing the technical chain of cause and effect; explainability is about presenting that chain — or a relevant summary of it — in a form that's interpretable by humans who need to understand, trust, or challenge an AI system's output. Complete traceability is a prerequisite for explainability but doesn't automatically produce it — a full lineage graph with hundreds of nodes is traceable but not, in that raw form, explainable to a non-technical stakeholder.

Explainability Requirements by Stakeholder

Technical / Engineering (Debugging)

Requires the most complete, granular lineage — full trace trees, exact prompt text, retrieved document IDs, model version strings. This audience can interpret raw lineage graphs and trace data directly; the requirement is completeness (Sections 1-6) and queryability, not simplification.

Business Users (Understanding 'Why Did the System Do This')

Requires lineage information translated into business terms — not 'retrieved chunk_id=4471 with cosine similarity 0.83' but 'based on the return policy document updated last month'. This translation requires the semantic layer connection (companion Architecture Evolution report, Section 7) — mapping technical lineage entities to business-meaningful names and descriptions, which depends on metadata quality (documentation, naming) that's often inconsistent across the six layers.

Auditors / Regulators (Compliance Verification)

Requires lineage presented as formal evidence — often in specific formats (e.g., EU AI Act technical documentation templates) with attestation that the lineage record is complete and accurate for the scope being audited. This is the use case most directly served by the unified evidence-collection architecture discussed in the Compliance Reporting section, and the use case most exposed by lineage gaps (a gap means the attestation of completeness cannot be made).

End Users (Understanding/Contesting a Decision That Affects Them)

Requires the most simplified, individual-decision-scoped explanation — 'your request was declined because X', potentially with a path to request human review. This is the explainability requirement most directly tied to sector-specific regulations (RBI/MAS FEAT principles) and is arguably the highest-stakes explainability use case, since it directly affects an individual's outcome and their ability to contest it — yet it depends on the same underlying lineage (Layers 1-5) that's least complete for AI/agent-based decisions.

Explainability Techniques and Their Lineage Dependencies

- **RAG citation/source display:** showing users 'this response was based on these documents' is the most widely-deployed explainability technique for generative AI, and depends directly on Layer 4 lineage (which documents were retrieved) — but per Section 4, this typically shows the immediate retrieval result without connecting to whether that source document is current/correct (Layer 1), meaning the citation itself can be explainable while still being misleading if the cited source is stale
- **Chain-of-thought / reasoning trace display:** showing an agent's intermediate reasoning steps provides explainability for Layer 5 (why did the agent take this action), but as noted in Section 5, reasoning traces are not always retained or exposed, and even when available, raw chain-of-thought is often not in a form directly meaningful to business users or end users without further translation
- **Feature importance / SHAP-style explanations:** for traditional ML components within an AI system (Layer 3), feature importance techniques explain a specific prediction in terms of which input features most influenced it — this is a mature explainability technique but applies to Layer 3 (traditional ML models), not directly to Layer 4-5 (LLM/agent outputs), creating an explainability gap for systems where traditional ML components feed into LLM-based agent decisions: feature importance can explain the ML model's contribution, but not how the agent weighted/used that contribution
- **Counterfactual explanations:** 'this decision would have been different if X had been different' — valuable for end-user explainability (especially for adverse decisions, where understanding what would change the outcome is often more actionable than understanding the full causal chain) but requires the ability to re-run the decision process with modified inputs, which for LLM-based decisions runs into the non-reproducibility limitation discussed in Section 4 — a counterfactual re-run may produce a different output not because of the changed input but due to inherent LLM output variability, undermining the counterfactual's validity

The Explainability Ceiling Imposed by Lineage Gaps

A central conclusion of this report: explainability for AI systems cannot exceed the completeness of the underlying lineage. No explainability technique — however sophisticated its presentation — can explain a connection that the lineage graph doesn't contain. For Layer 4-6, where lineage connections to upstream layers are frequently absent (Sections 4-6), the practical effect is an 'explainability ceiling': an organization can present RAG citations (Layer 4 lineage that exists) but cannot, without closing the Layer 4-to-Layer 1 gap, explain whether the cited source itself was reliable — the explanation looks complete to the recipient ('here's the source') while omitting a dimension (source currency/reliability) that the lineage graph simply doesn't track. This creates a risk that's distinct from 'no explanation provided': a partial explanation that appears complete can be more problematic for trust and compliance than an explicit 'we cannot fully explain this' — because it doesn't prompt further verification.

Recommendation: Explainability interfaces (citation displays, reasoning traces) should explicitly indicate the boundary of what's been verified versus what's been retrieved-but-not-verified — e.g., distinguishing 'this response cites Document X (last verified current as of [date])' from 'this response cites Document X (currency not verified)' where Layer 4-to-Layer 1 lineage doesn't extend to a freshness check. This is a presentation-layer mitigation for an underlying lineage gap — it doesn't close the gap, but it prevents the gap from being silently presented as completeness, which this report identifies as the more acute risk.

Lineage for Multi-Agent AI Systems

Framing: Single-agent lineage (Section 5) is a hard problem that remains substantially unsolved. Multi-agent lineage is not simply 'single-agent lineage, N times' — it introduces structural properties (concurrency, agent-to-agent communication, shared memory, emergent coordination) that change the shape of the lineage problem from a trace (a sequence with branches) to a graph (a network of interacting traces) — closer in structure to the knowledge graphs of Section 6 than to the distributed traces of Section 5, even though the underlying instrumentation (OpenTelemetry-derived tracing) was designed for the latter.

How Multi-Agent Architectures Change the Lineage Problem

From Sequential Traces to Communication Graphs

A single agent's execution, however complex, can be represented as a trace tree — a single root (the initiating request) with branches for sub-tasks/tool calls, ultimately converging on a response. Multi-agent systems using protocols like A2A (companion Operational Excellence report, Part 17) involve multiple agents, each potentially with their own trace tree, communicating with each other — the overall lineage structure is a graph of interconnected trace trees, where edges represent agent-to-agent messages/task delegations. Current trace-based tooling can capture each agent's individual trace but typically does not natively represent the cross-agent graph structure connecting them — each agent's trace may reference a 'delegated task ID' from another agent, but reconstructing the full multi-agent graph from these references requires custom correlation logic, since no standard exists for representing this graph structure directly.

From Sequential Execution to Concurrent Execution

Single-agent traces are largely sequential (even with parallel tool calls, there's a single overarching task with a defined start and end). Multi-agent systems may have multiple agents operating concurrently on related or overlapping tasks — lineage must capture not just 'what happened' but 'what was happening concurrently', since concurrent operations on shared resources (e.g., two agents both updating the same agent memory entity, per the companion Architecture Evolution report's Section 10) can produce results that depend on timing/ordering in ways that a purely sequential lineage record cannot represent.

Shared Memory as a New Lineage Surface

When multiple agents read from and write to a shared memory store (companion Architecture Evolution report, Section 10), the memory store itself becomes a lineage surface analogous to a shared database table in traditional data lineage — but with additional complexity: a memory write by Agent A might be read by Agent B and influence Agent B's subsequent actions, creating a lineage edge (A's write → B's read → B's action) that exists entirely within the memory store's read/write history, separate from either agent's individual execution trace. Capturing this requires the memory store itself to maintain lineage-relevant read/write logs — a requirement not addressed by current agent memory platforms (per the companion report's Section 10, these platforms have not yet addressed even basic governance requirements, let alone lineage).

Emergent Coordination Without a Single Point of Lineage Capture

In multi-agent systems where coordination emerges from agent-to-agent negotiation (rather than being centrally orchestrated, as in single-controller frameworks like LangGraph), there may be no single component with visibility into the overall coordination pattern — each agent has visibility into its own interactions, but the emergent system-level behavior (e.g., three agents collectively arriving at a decision through several rounds of negotiation) exists only as the union of each agent's partial view. Lineage for emergent multi-agent behavior therefore requires either a centralized observability layer that all agents report to (creating a single point of lineage capture, but also a single point of failure and a potential bottleneck) or a decentralized approach where lineage records themselves carry enough cross-referencing information to be reassembled post-hoc — the latter being closer to how distributed systems tracing solved an analogous problem for microservices, but not yet adapted for agent-to-agent communication specifically.

What a Multi-Agent Lineage Model Would Need to Capture

Extrapolating from the six-layer model and the structural changes above, a multi-agent lineage model — which does not yet exist in mature form as of 2026 — would need to represent at minimum:

- **Agent identity and provenance:** for every action, which specific agent (with a stable identity per the companion Operational Excellence report's Part 14) performed it, including agents that are themselves dynamically instantiated (e.g., a coordinator agent spawning worker agents for sub-tasks)
- **Inter-agent message lineage:** for every agent-to-agent communication (A2A or equivalent), what was communicated, by which agent, to which agent(s), and how did the receiving agent's subsequent actions relate to the received message — essentially, Layer 5 agent lineage extended with a 'sender/receiver' dimension
- **Shared-resource read/write lineage:** for every read or write to shared memory, shared knowledge graph segments, or shared tool/data resources, which agent performed the operation, what was the state before/after, and which subsequent operations by other agents depended on this state
- **Temporal/concurrency metadata:** sufficient timing information to reconstruct the actual order of operations across agents, including identifying genuinely concurrent operations (where ordering is not meaningful) versus causally-ordered operations (where ordering matters for understanding cause and effect)
- **Aggregate/emergent outcome attribution:** for a final multi-agent system outcome, the ability to attribute the outcome to the contributing agents' individual actions in a way that's meaningful for accountability — analogous to how Layer 2 feature lineage attributes a model's behavior to specific features, but for agent contributions to a collective outcome

Implications for the Six-Layer Model

Multi-agent systems don't add a 'seventh layer' so much as they multiply and interconnect Layer 5 (agent lineage) — and, by extension, every layer that Layer 5 depends on. A single multi-agent task may involve dozens of individual agent executions, each with its own Layer 1-4 dependencies, communicating via Layer 5 extensions, and potentially reading/writing Layer 6 knowledge structures concurrently. The lineage graph for a single multi-agent task could therefore be substantially larger and more interconnected than the lineage graph for an entire traditional data pipeline run — a scale and structural shift that has implications for lineage storage (graph databases, per the companion Architecture Evolution report's Section 6, are likely necessary rather than optional for multi-agent lineage at scale), query patterns (impact/root-cause analysis across a much larger, more interconnected graph), and — most acutely — for the explainability ceiling discussed in the prior section, which becomes substantially lower (more partial) for multi-agent outcomes than for single-agent or traditional ML outcomes, simply because there are more potential gaps in a larger graph.

Forward-Looking Assessment

As of 2026, multi-agent lineage should be considered an open research and tooling problem rather than a solved or near-solved one. Organizations deploying multi-agent systems in production — particularly for use cases with compliance exposure (Table 4) — face a lineage maturity gap larger than any single layer discussed elsewhere in this report, because multi-agent lineage depends on Layer 5 maturity (itself the least mature single-agent layer) plus entirely new cross-agent lineage concepts with no current standard. Organizations in this position should: (1) treat multi-agent lineage as a custom-engineering problem requiring dedicated investment, not an off-the-shelf capability; (2) prioritize centralized observability approaches (a shared tracing/lineage backend all agents report to) over fully decentralized approaches, given the absence of standards for the latter; (3) explicitly scope which multi-agent interactions are considered 'in scope' for lineage capture based on compliance and risk exposure (Table 4), rather than attempting comprehensive capture of all agent-to-agent communication, which at scale would itself become an unmanageable volume of lineage data; and (4) monitor the OpenInference/OpenTelemetry ecosystem (Platform Comparison section) for emerging multi-agent semantic conventions, as this is the most likely source of eventual standardization given its trajectory for single-agent (Layer 5) lineage.

Closing Synthesis: This report's six-layer model describes a lineage landscape where completeness decreases as data moves from raw tables toward autonomous agent decisions — and multi-agent systems sit at the far end of this gradient, depending on the least mature layer (agent lineage) while introducing structural requirements (communication graphs, concurrency, shared memory, emergent coordination) that exceed even Layer 5's current scope. The practical consequence for enterprises is that the rapid adoption of multi-agent architectures (companion Operational Excellence report, Part 17) is significantly outpacing the lineage and traceability infrastructure needed to govern, debug, and demonstrate compliance for these systems — making lineage investment, specifically targeted at the Layer 4-6 connections and agent-to-agent communication capture described in this report, one of the highest-leverage and most time-sensitive infrastructure investments for any enterprise scaling multi-agent deployments through 2026 and beyond.